pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Model Files and Drift

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: model files and drift

Today

1. Course 4 day 11's task, served
2. A model file and its card
3. The model behind an endpoint
4. The held-out days arrive as requests
5. Drift: the inputs against the window
6. Flags, and the error by quarter
7. What drift does not say

Two windows, split by date

Development years

2015-01-02 to 2022-12-30, 2,001 days

fits the model

Held-out years

2023-01-03 to 2026-10-02, 939 days

arrive only as requests

A model stops changing the day it is saved. Its inputs do not.

What the service returns

The fitted same-day 30 Yr yield: for the same date as its two inputs, with its window.

Allowed names

the fitted same-day 30 Yr yield

inputs outside the training window

the drift table

Never called

a forecast of rates

a signal or an outlook

a prediction

Every response carries the training window: 2015-01-02 to 2022-12-30.

Course 4 day 11's task, served

Inputs and output

The same day's 2 Yr and 10 Yr par yields in; the fitted same-day 30 Yr yield out.

```
DecisionTreeRegressor(max_depth=4, random_state=SEED).fit(development[FEATURES], development[TARGET])
```

16

leaves: at most this many different values, whatever the input

9.2 bp

mean absolute error on the development years only

2,001

days fitted on, 2015 to 2022

Course 4: a tree cannot go beyond what it has seen. Today it happens in a service.

How a model file works

The problem	A fitted model is an object in one process; the server is another process, often on another machine
The method	joblib.dump writes the object as a pickle, arrays included; joblib.load rebuilds it, which is why opening the file runs code
The card	save_model writes model_card.json beside the file: window, rows, versions, and the file's SHA-256 fingerprint
The check	load_model reads the card, computes the fingerprint again, refuses a changed file, and only then runs joblib.load
What comes out	Two files: model.joblib, never committed, and model_card.json, text you read, review, and commit

A model file and its card

```
"name": "the fitted same-day 30 Yr yield"
"features": ["2 Yr", "10 Yr"]   "target": "30 Yr"
"training_window": "2015-01-02 to 2022-12-30"
"rows": 2001
"sha256": "9a6d8fdd1fafbc3468865cec..."
"versions": python, scikit-learn, joblib, numpy
```

model.joblib

3,633 bytes

a pickle: loading it runs code

never committed

```
refused, ValueError: model.joblib is not the file its card describes: its SHA-256 differs; do not load it
```

load_model checks the SHA-256 against the card before joblib.load runs.

Never load a model file someone else made

joblib's own documentation

"joblib.load relies on the pickle module and can therefore execute arbitrary Python code."

joblib.readthedocs.io/en/stable/generated/joblib.load.html, read 2026-10-12

Python's own documentation

"The pickle module is not secure. Only unpickle data you trust."

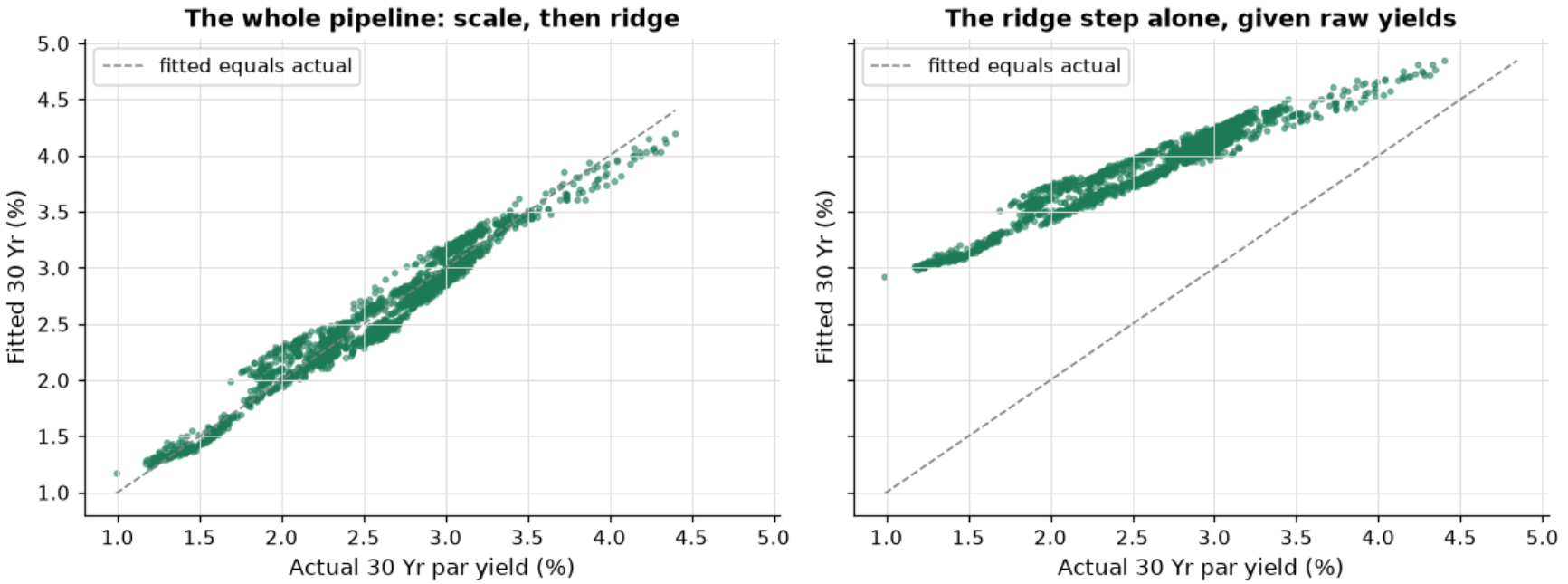
docs.python.org/3/library/pickle.html, read 2026-10-12

The SHA-256 proves a file is yours, unchanged. It never makes a stranger's file safe.

How a pipeline works

The problem	Most models have a scaler, an imputer, or an encoder in front, fitted on the training rows; the model step is fitted on what comes out
The method	A Pipeline holds the steps and the model as one object: fit runs them in order, predict runs the same fitted steps first
The rule	The whole pipeline goes in the file, never the model step alone: a step alone expects inputs already through the others
You control	The steps and their order
What comes out	One file that answers raw inputs the way the notebook did

The pipeline against the step alone



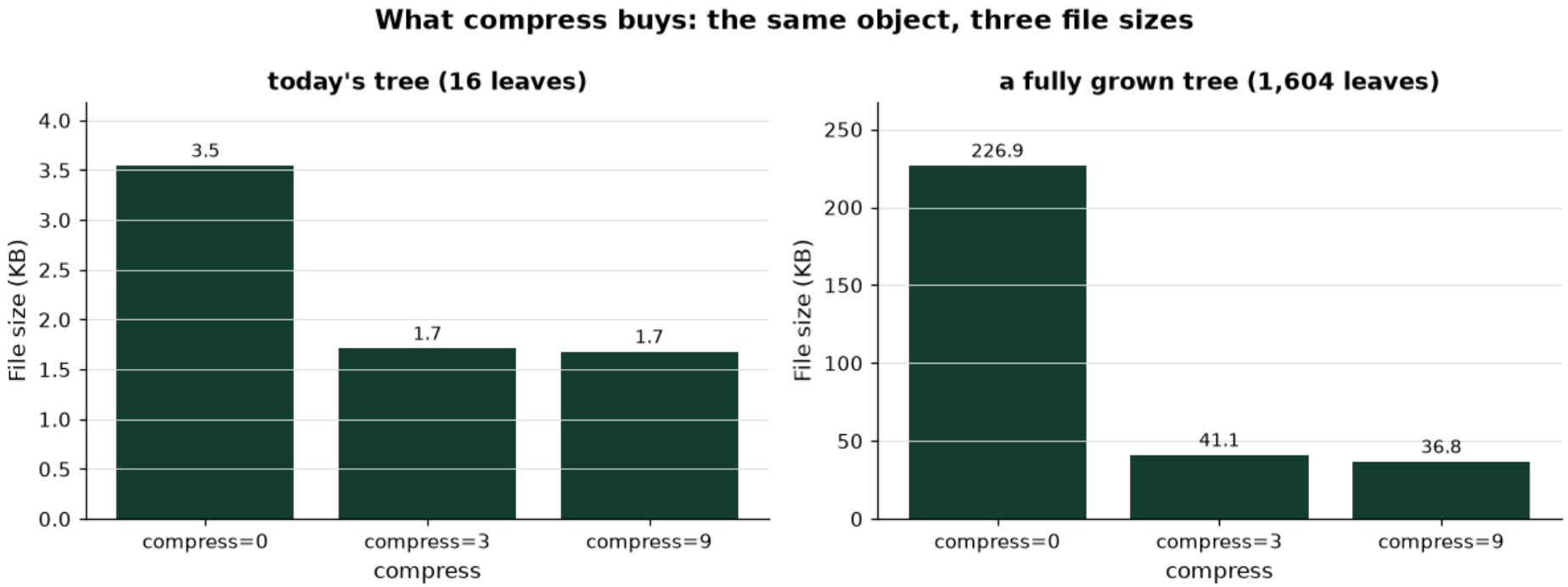
The whole pipeline: 9.2 bp. The ridge step alone, given raw yields: 130.4 bp. Nothing is wrong with the weights; the step in front is missing.

Every setting of joblib

Function	Setting	Default	What it does	The course passes
dump	value	(required)	the object to write: the whole pipeline, never one step	the fitted model or pipeline
dump	file name	(required)	the path written to; a .z, .gz, .bz2, .xz, .lzma extension picks compression	model.joblib in the model's folder
dump	compress	0	0 to 9, True (3), or (method, level); 0 writes the pickle as it is	the default, 0
dump	protocol	None	the pickle protocol; None is pickle's default for this Python	the default
load	file name	(required)	the path read	the file the card names, after the check
load	mmap_mode	None	None reads arrays in; r, r+, c, w+ map them from disk (uncompressed only)	the default, None
load	ensure_native_byte_order	'auto'	auto: match the machine's byte order when arrays are read in	the default

7 settings in the installed joblib, two required. The course passes the path and leaves the rest at its default: compress is the one worth a choice.

The file at three compress levels



Today's tree: 3,633, 1,754, 1,720 bytes. The fully grown tree: 226.9, 41.1, 36.8 KB. Level 3 does most of the work; the course stays at 0 for a file this size.

How a model service works

The problem	A model file on a disk answers nobody; a service puts a route in front of it
Loading	model_app reads the card and loads the model once through load_model: a changed file stops the service at its start
Two routes	GET /health names the model it holds; POST /fitted-30y takes a CurveRequest, checked by pydantic before your code runs
Each answer	A one-row frame, predict, each input against the training range, one line to the input log, the window in the reply
What comes out	An app the test client runs here and uvicorn runs on a server, JSON for JSON

The model behind an endpoint

```
200 {
 "name": "the fitted same-day 30 Yr yield",
 "as_of": "2022-12-30",
 "fitted_30y_pct": 3.9053846153846163,
 "training_window": "2015-01-02 to 2022-12-30",
 "inputs_outside_training_range": []
}
```

POST /fitted-30y

as_of: the date the yields are from

y2_pct, y10_pct: strict numbers

every field required

the model loads once, checked

an input log: date, inputs, value

```
422 [(['body', 'y10_pct'], 'Field required')]
requirements_for: ['fastapi==0.143.0', 'pandas==3.0.6', 'pydantic==2.13.5']
```

requirements_for reads imports. The model file also needs scikit-learn and joblib. A new model is a new folder beside the old: model_v1_1.

The held-out days arrive as requests

939

requests, one per held-out day, in date order; every one status 200

4

distinct fitted values returned

16

leaves the tree could return

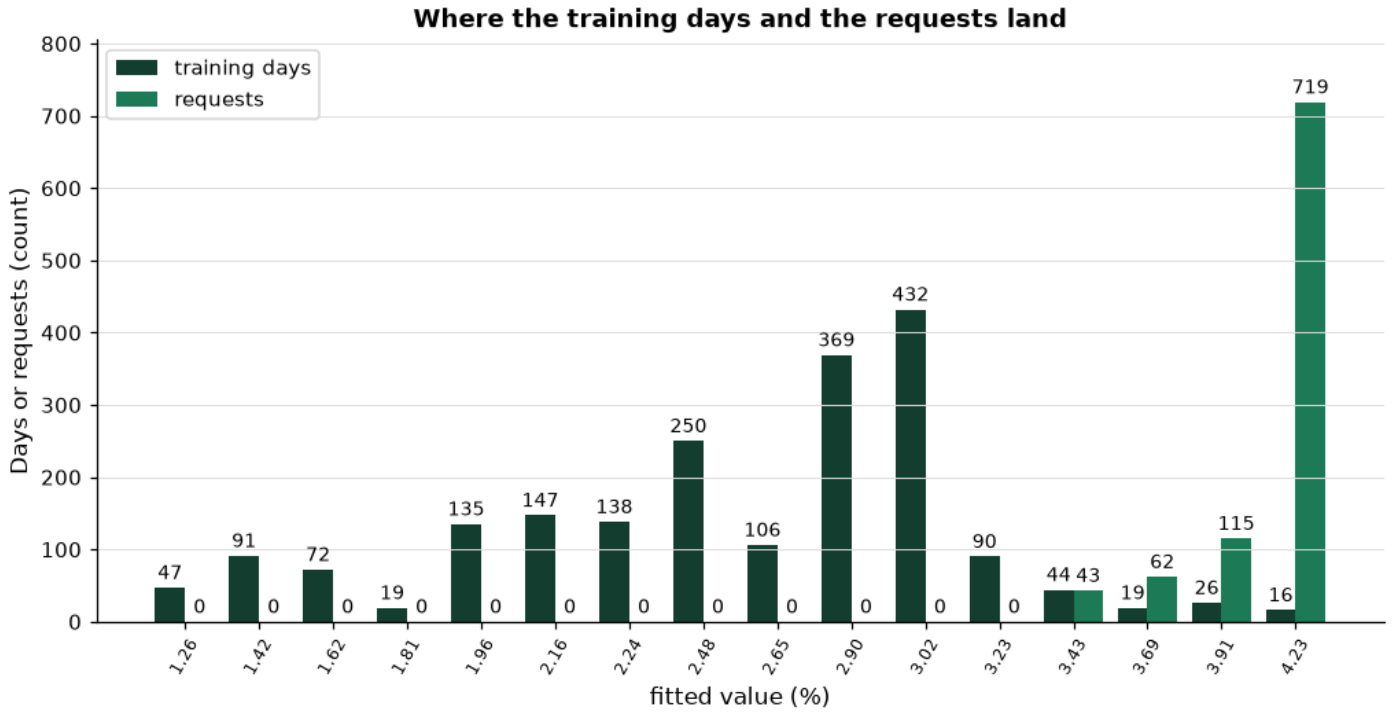
The input log

One JSON line a request: the date, the 2 Yr, the 10 Yr, the value returned.

Nothing about who sent it. Day 12's request log still holds no body.

A tree returns one of its leaves: on these days, only a few of them.

Where the days land



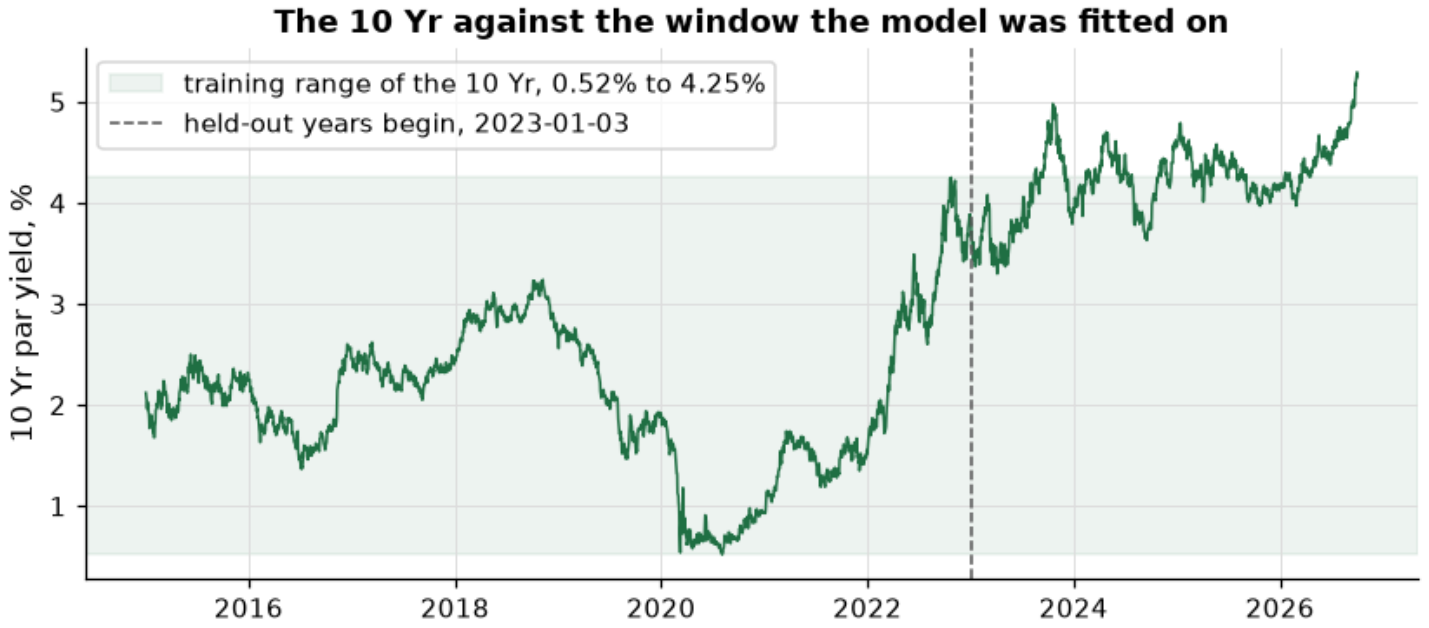
The days it was fitted on spread over all 16 values; the requests use only 4.

How drift_table works

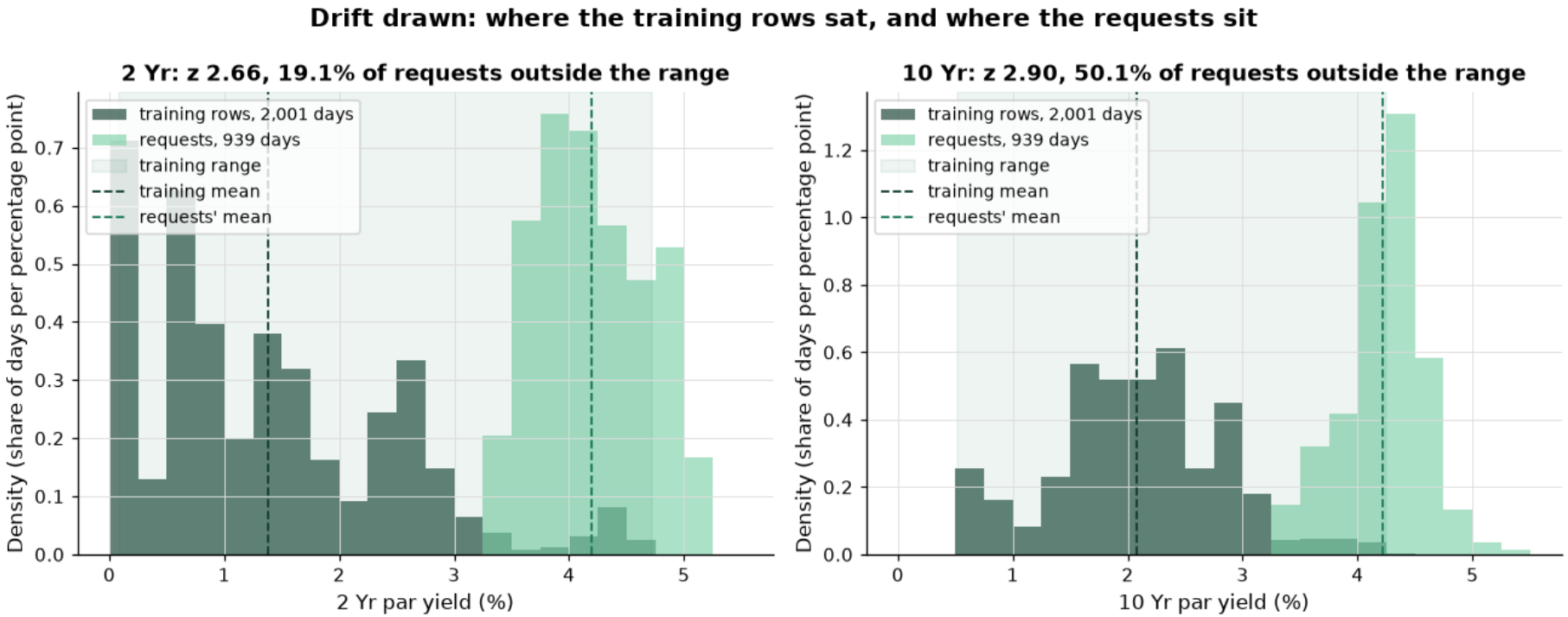
The problem	A service never sees the 30 Yr when it answers, so the first warning has to come from the inputs alone
The reference	The training rows' mean, standard deviation (ddof 1), minimum, and maximum, and the requests never enter them
Two readings	The z of the recent mean, in training standard deviations; the outside share, requests below the minimum or above the maximum
You control	Which rows are the reference, which are recent, and which features to compare
What comes out	One row per input with seven numbers, computed from inputs alone, so a service can run it every day

Drift: the inputs against the training window

	train_mean	train_std	train_min	train_max	recent_mean	z	outside_share
2 Yr	1.379	1.061	0.09	4.72	4.201	2.660	0.191
10 Yr	2.079	0.739	0.52	4.25	4.218	2.895	0.501

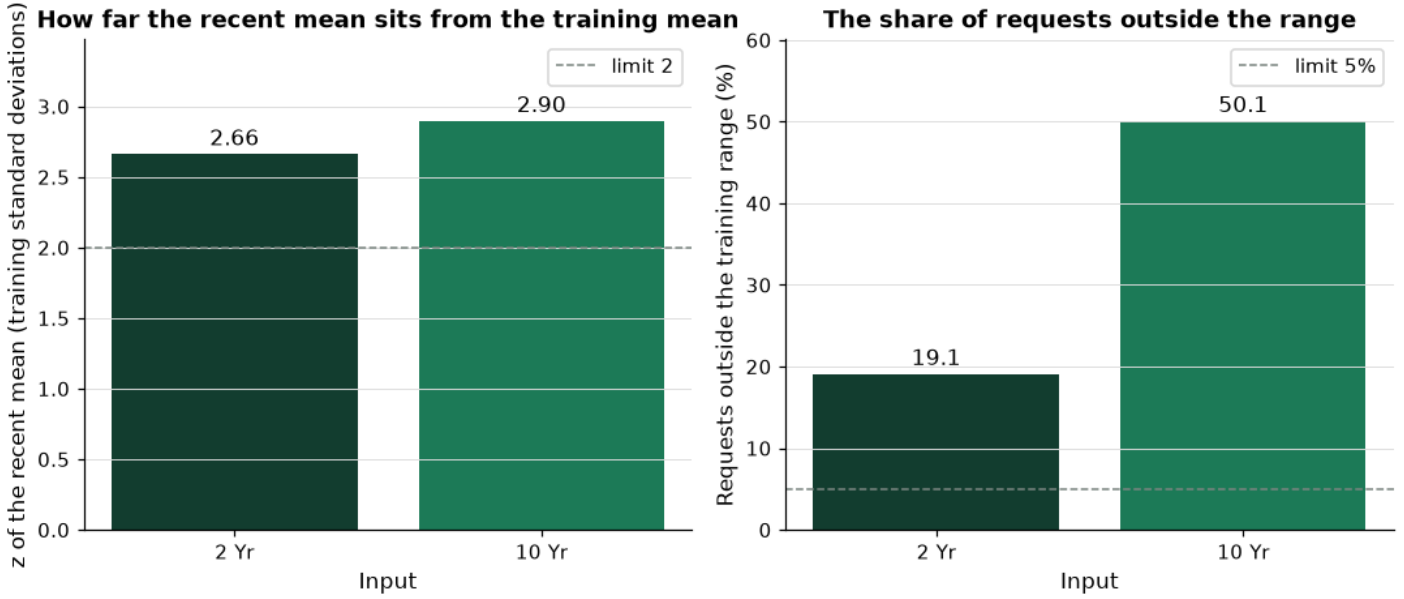


Drift drawn



The gap between the dashed lines is the z; the light bars past the band are the outside share: 179 and 470 requests above the maximum, 0 below.

The drift against the limits



Input	z	Outside share
2 Yr	2.660	0.191
10 Yr	2.895	0.501

Both inputs sit over both default limits: a z above 2, and more than 5% outside the range.

How drift_flags works

The problem	A drift table is two numbers per input, and someone has to say when a number is too large
The rule	Two limits written down before the data is seen: z_limit (2) and outside_limit (5%); over either, the input is flagged
With the flag	The rule in words in attrs["rule"], so a printed flag always travels with the limits that set it
You control	The two limits; a change to either is a change a reviewer can diff
What comes out	The same table with a flag column, True or False per input, and the rule as a sentence

Flags, and the error by quarter

Rule: flag when |z| > 2 or more than 5% of recent rows are outside the training range

29.4 bp

415 days with both inputs inside the training range

54.2 bp

524 days with an input outside it

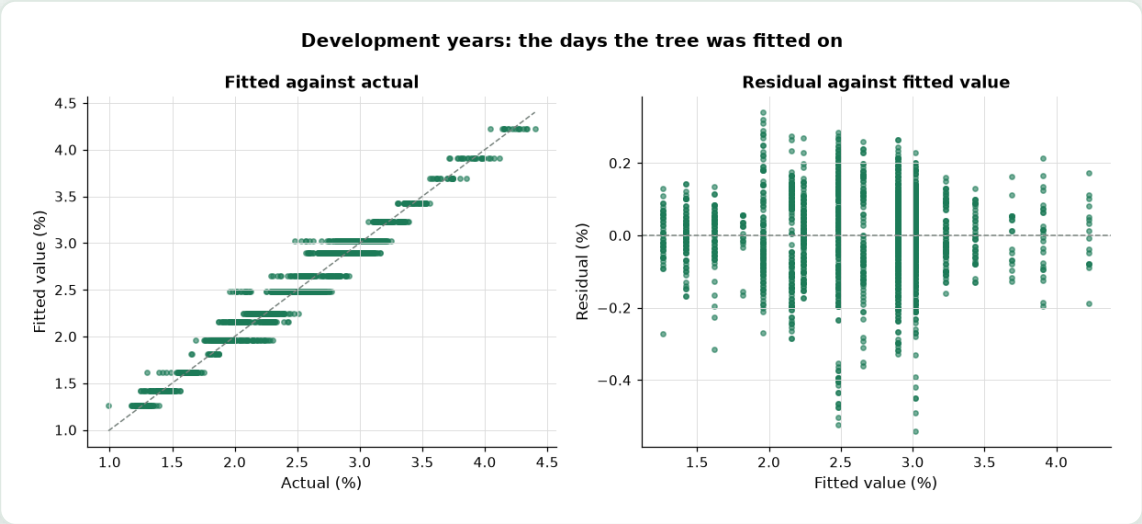
4.23%

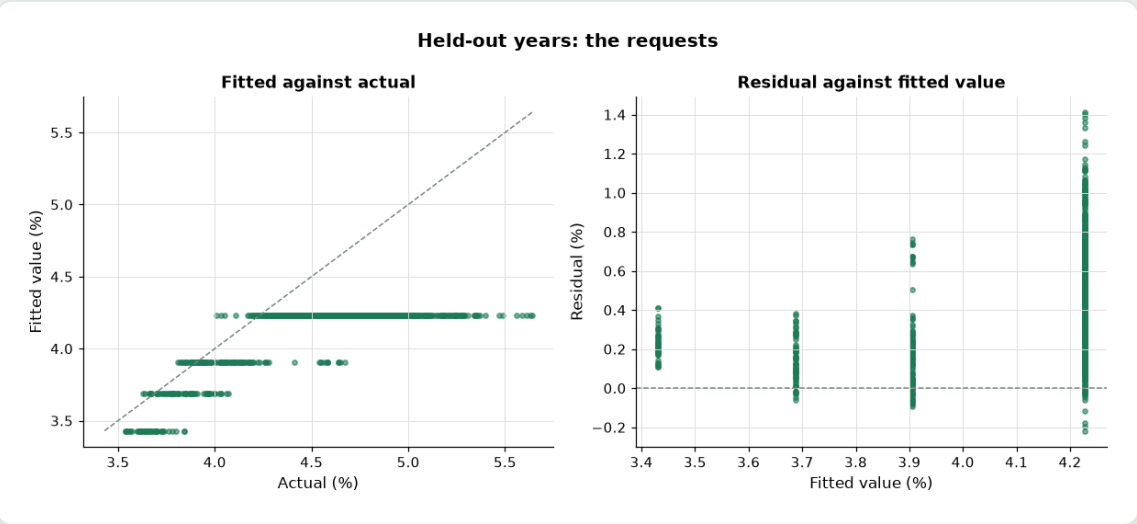
the highest value returned; the held-out 30 Yr reached 5.64%

```
quarter   days  outside_share  mae_bp
2023Q1    62           0.16    9.53
2023Q2    63           0.08   17.85
2025Q4    62           0.00   53.26
2026Q2    63           1.00   73.25
2026Q3    64           1.00   99.95
```

The error grows where the inputs leave the range. Inside the range is not a guarantee.

The tree on the requests

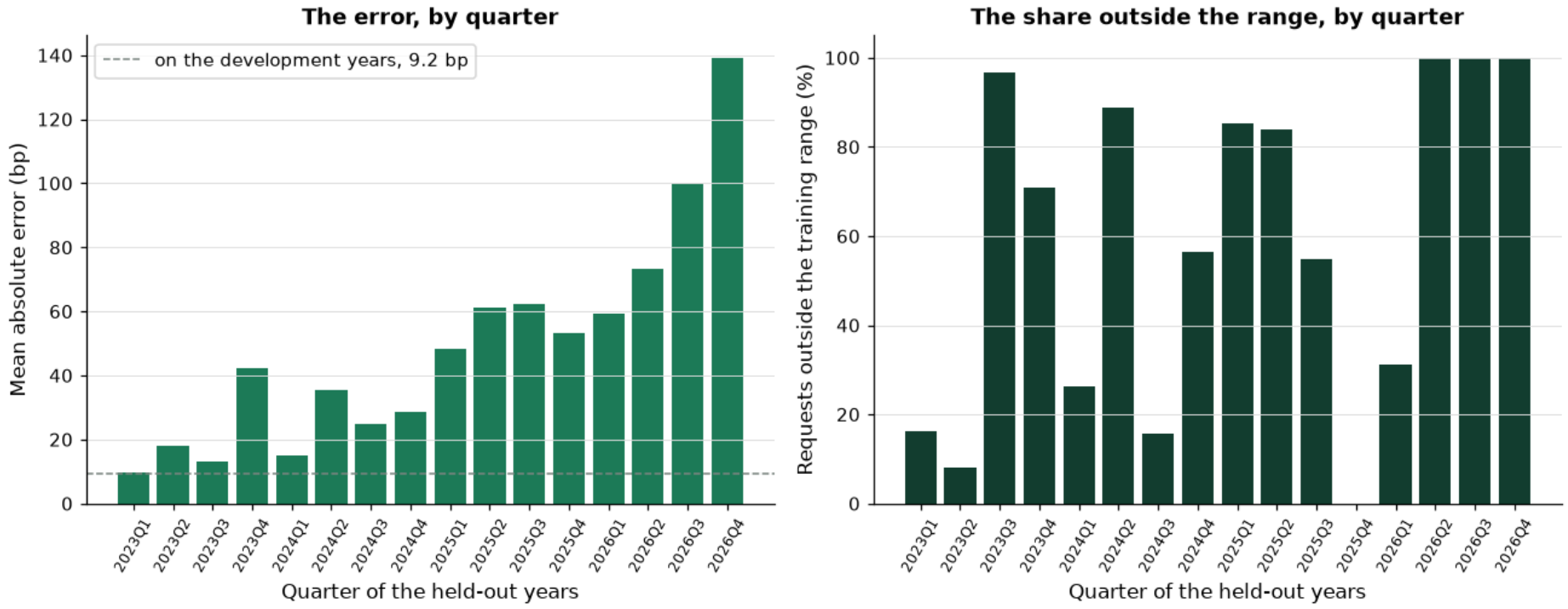




Rows	RMSE	MAE	MAPE (%)	R-squared
development years	0.1195	0.0921	3.7599	0.9598
held-out years	0.5163	0.4325	9.0776	-0.4292

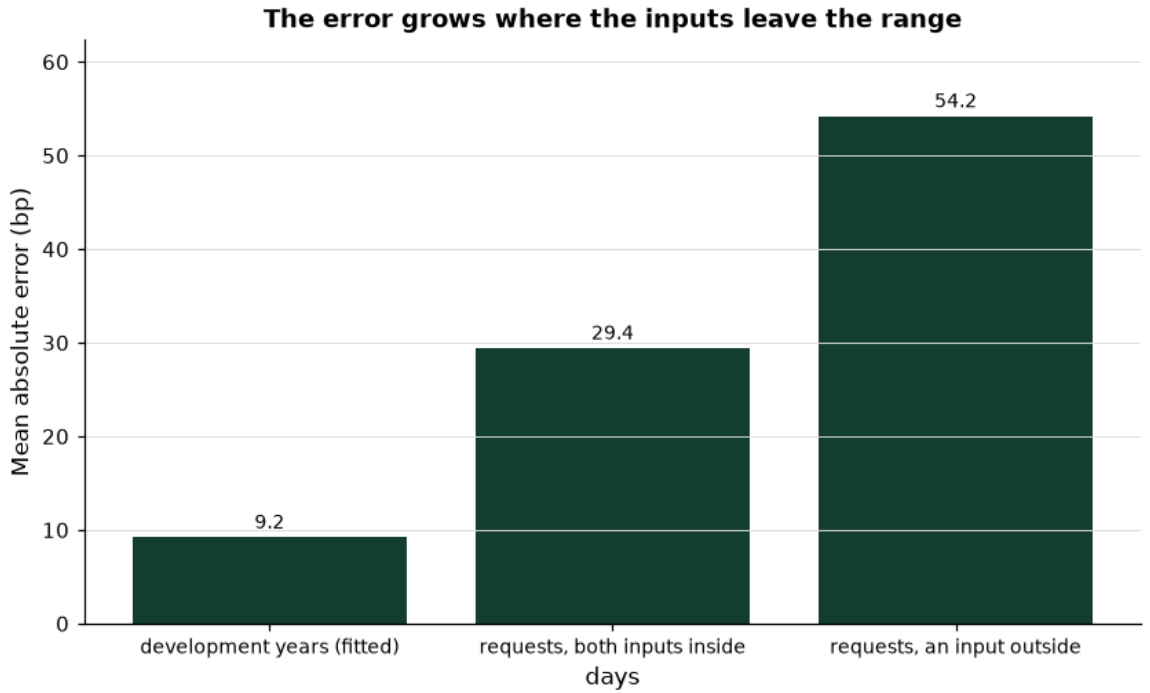
On the requests the fitted values stop at a ceiling, and R-squared is below zero. RMSE and MAE in percentage points.

The error by quarter



One quarter has nothing outside the range, and its error is still well above the first quarter's: inside the range is not a guarantee.

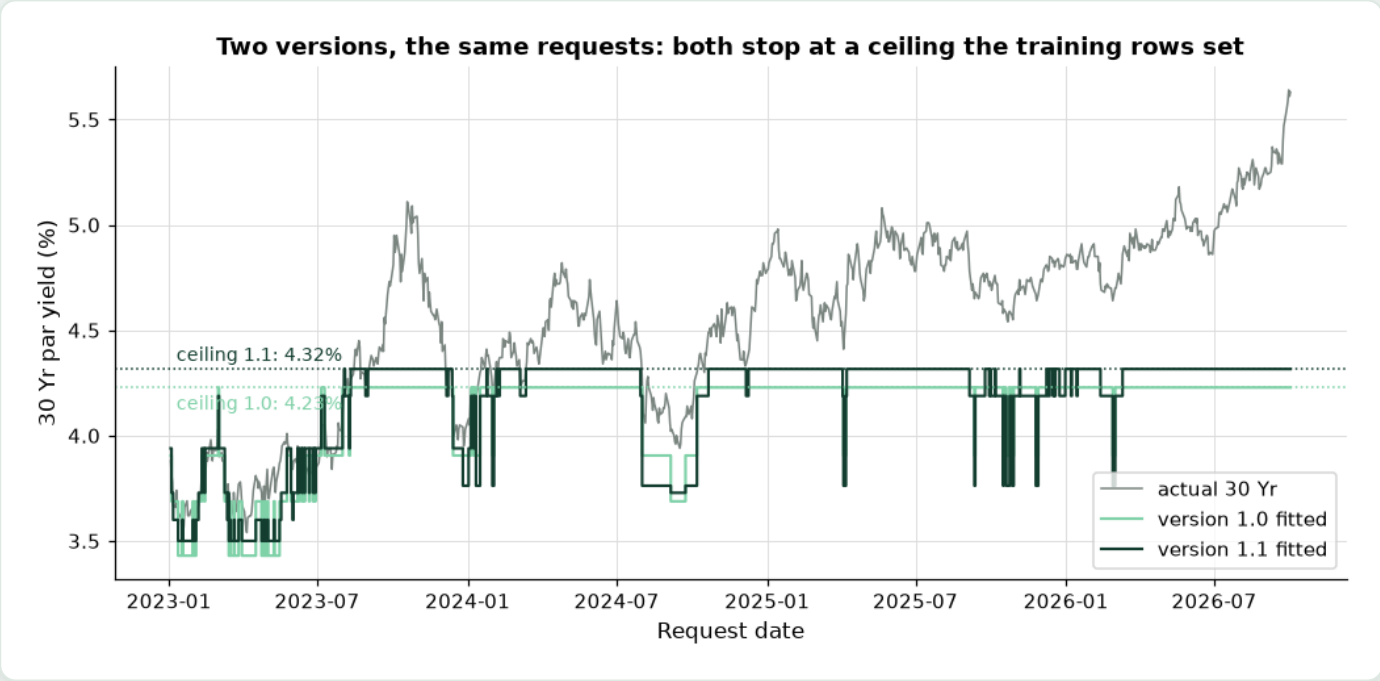
Inside and outside the range



Days	Count	Error (bp)
fitted on	2,001	9.2
inside range	415	29.4
outside range	524	54.2

The error grows where the inputs leave the training range.

Two versions on the same requests



Version	Leaves	Ceiling (%)	Error (bp)
1.0 (depth 4)	16	4.23	43.25
1.1 (depth 5)	32	4.32	38.72

Ceilings 4.23% and 4.32%, errors 43.2 and 38.7 bp: a different model, not a different kind of answer. Whether to serve 1.1 is section 13.7's decision.

What drift does not say

Not why

the inputs moved: a question about the economy

Not next quarter

no view on rates, no forecast

Not wrong today

a flagged input can land in a leaf that fits

Not a test passed

pytest checks code, not the world

What to do about drift is a decision for the people who use the service.

Excel and Git

Excel side by side (tested in Excel)

`=STANDARDIZE(AVERAGE(recent!C2:C940),AVERAGE(train!C2:C2002),STDEV.S(train!C2:C2002))`	2.895069
`=STANDARDIZE(..., STDEV.P(train!C2:C2002))`	2.895793
`=COUNTIFS(recent!C2:C940,">"&MAX(train!C2:C2002))+COUNTIFS(...,"<"&MIN(...))`	470

```
.gitignore:6:*.joblib   model/model.joblib
under model/ on the remote: ['model/model_card.json']
any .joblib file on the remote: no
```

The card is committed and pushed. The model file is ignored, with the reason beside the line.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the reference pools the training rows and the requests: the 10 Yr's z falls to 1.23, and no flag.

Code that runs is not the same as code that is right.

What you can do now

- Save a model file with a card; load it only if its SHA-256 matches
- The whole pipeline in the file, never the model step alone
- Never load a joblib or pickle file someone else made
- Measure drift against the training rows only
- Commit the card, never the file

Week 3, Day 4 Secrets, limits, and a cost meter